The National University of Lesotho

Department of Mathematics and Computer Science

Faculty of Science and technology



CS4433: Software engineering

Task: Software Design Document

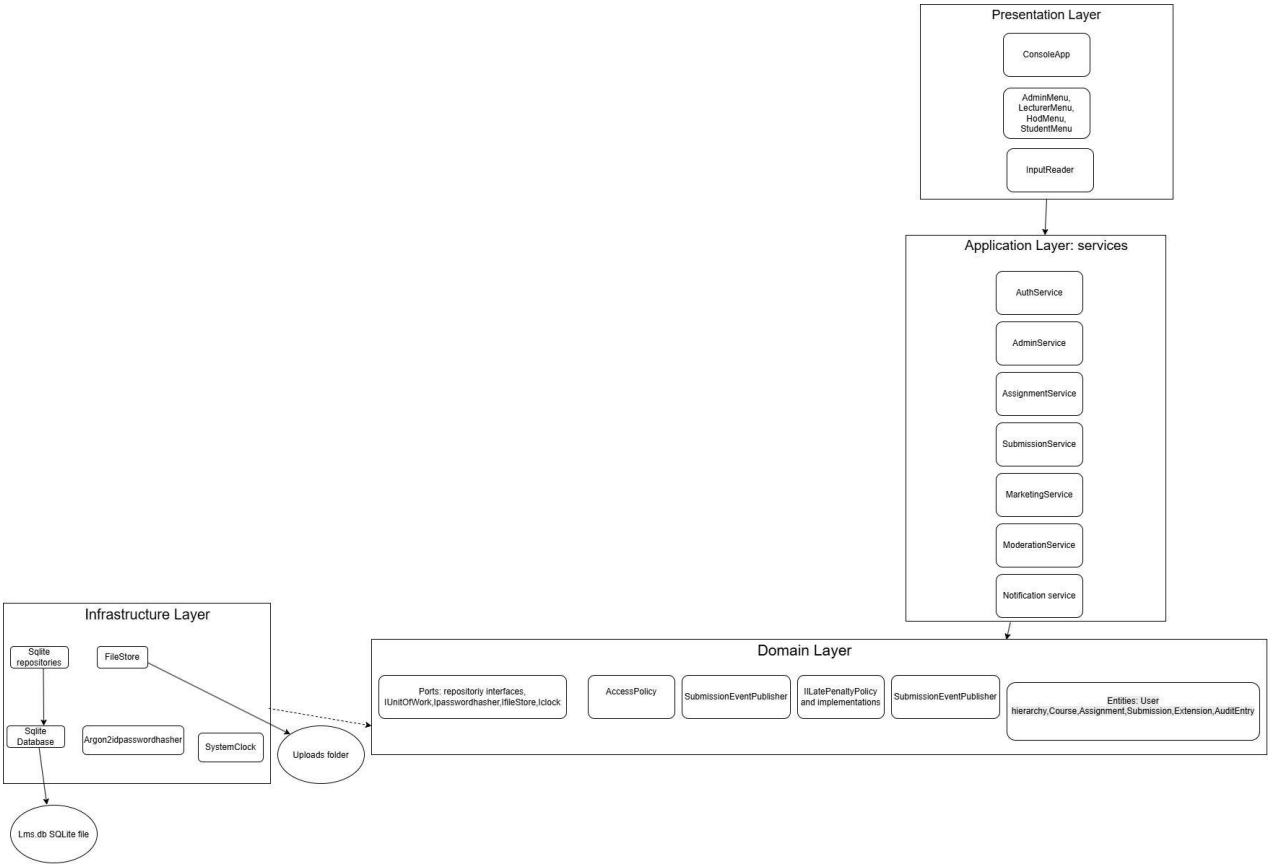
Due: 19th October 2026

Participant:

Student Number	Surname, initials	Major
202322798	T'soeu, D. T	B. Sc. IS

1. Architecture

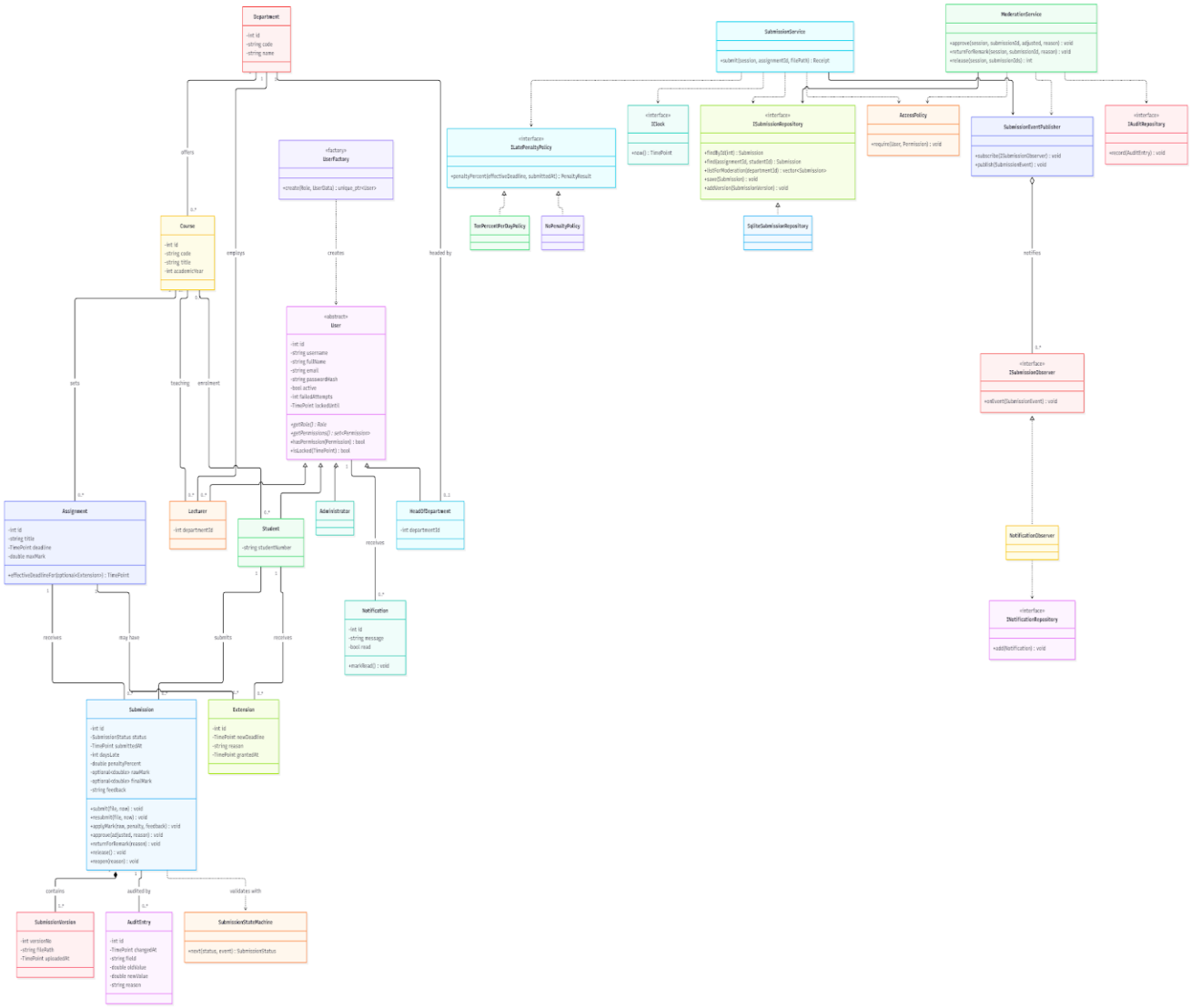
- **Style selected:** Layered architecture with dependency inversion, in four layers: Presentation, Application (services), Domain, Infrastructure. The domain owns the repository interfaces and the infrastructure implements them.



- **Justification:** The hard part of this system is its rules: penalties, extensions, the moderation gate and role permissions. A domain layer with no I/O lets me test them directly. It also keeps the UI replaceable, suits one developer over seven weeks, and works offline.

- **Alternatives rejected:**
    - MVC: it is built for several live views of one model, a console app has one view, and controllers tend to absorb business rules.
    - Client-server: it needs a reliable network and a server that non-IT staff would have to maintain. SQLite is also an embedded engine, not a network server.
    - Single-layer, with menus calling SQL directly: the rules would be untestable and impossible to audit.

2. Class Diagram



Design patterns

I apply four patterns where each one removes a real problem, and I rejected two others on purpose.

Strategy (late penalties). The HoD's rule is a policy the school may change (five days, 5% per day, a different cap). ILatePenaltyPolicy with TenPercentPerDayPolicy as the v1.0 implementation keeps the rule out of SubmissionService. Adding a rule means adding a class, not editing existing code. Tests call the policy directly with chosen dates.

Repository (data access). The brief requires SQLite access only through a data-access layer. Repository interfaces hide SQL from services, let me run service tests against in-memory fakes, and make the audit and security story easy to defend: parameterised queries live in exactly one place.

Observer (notifications). When a submission is marked, returned, approved or released, other parts of the system need to react (tell the lecturer, tell the student). SubmissionEventPublisher emits events and NotificationObserver creates inbox messages. Services do not know who is listening, so email or a log observer could be added later. I deliberately did *not* use Observer for the audit log: an audit row must commit in the same transaction as the mark change, which a loosely coupled observer cannot guarantee.

Factory (user creation). Users are rebuilt from database rows and created by the administrator. UserFactory::create(role, data) returns a unique_ptr<User> of the right subclass, so no other code contains if (role == …) chains. A new role needs one new subclass and one factory case.

Rejected: State pattern for the lifecycle. With six states and about eleven transitions, a table-driven SubmissionStateMachine is shorter and easier to test than six state classes, and I can show the whole lifecycle on one screen. I would reconsider if states gained their own behaviour.

Rejected: Singleton for the database. A global connection hides dependencies and blocks test substitution. Instead, SqliteDatabase is created once in main and passed in.

SOLID principles

Principle	How the design applies it
Single Responsibility	Entities hold rules about themselves; each service handles one group of use cases; repositories only persist; menus only talk to users.
Open/Closed	New penalty rules, notification channels or roles are added by writing new classes (ILatePenaltyPolicy, ISubmissionObserver, a User subclass), not editing tested ones.
Liskov Substitution	Code that holds a User* works for every role. HeadOfDepartment does not inherit from Lecturer, because an HoD is not a drop-in lecturer: substituting one would grant or deny marking rights wrongly.
Interface Segregation	Repositories are small and per-aggregate. IClock, IPasswordHasher and IFileStore are separate so a service depends only on what it uses.
Dependency Inversion	Services depend on I… interfaces owned by the domain; concrete SQLite classes are wired in only in main.

Late-penalty rule with boundary cases

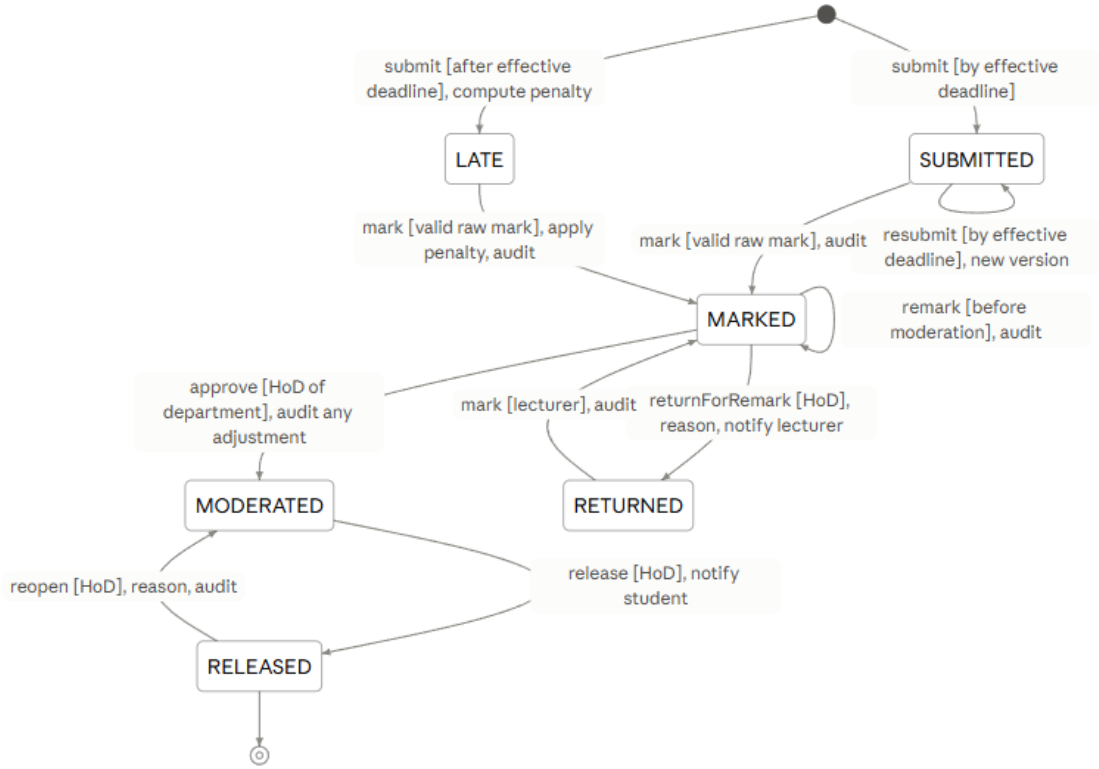
daysLate = ceil((submittedAt − effectiveDeadline) / 24 h), with no penalty when submittedAt ≤ effectiveDeadline. Penalty is 10% × daysLate for 1–3 days, and 100% beyond. The deduction is 10% of the *maximum* mark: finalMark = max(0, rawMark − penalty% × maxMark).

Worked example: deadline Friday 17:00, maximum mark 50.

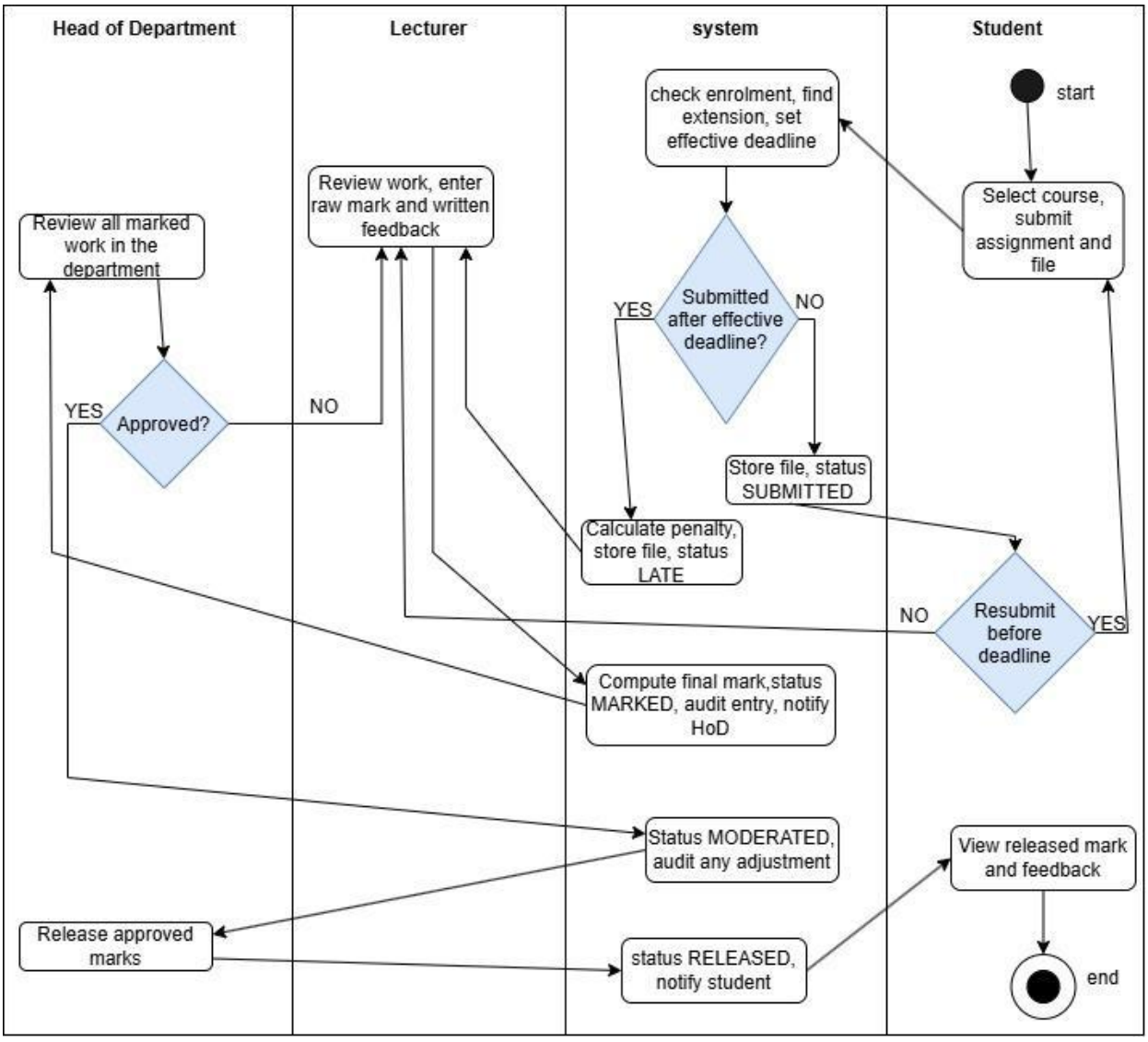
Submitted at	daysLate	Penalty	Raw 42 becomes
Fri 17:00:00	0 (on time)	0%	42
Fri 17:00:01	1	10% (5 marks)	37
Sat 17:00:00	1	10%	37
Sat 17:00:01	2	20% (10 marks)	32
Mon 17:00:00	3	30% (15 marks)	27
Mon 17:00:01	4	100%	0

3. Behavioral models

i) State machine diagram for the submission life cycle

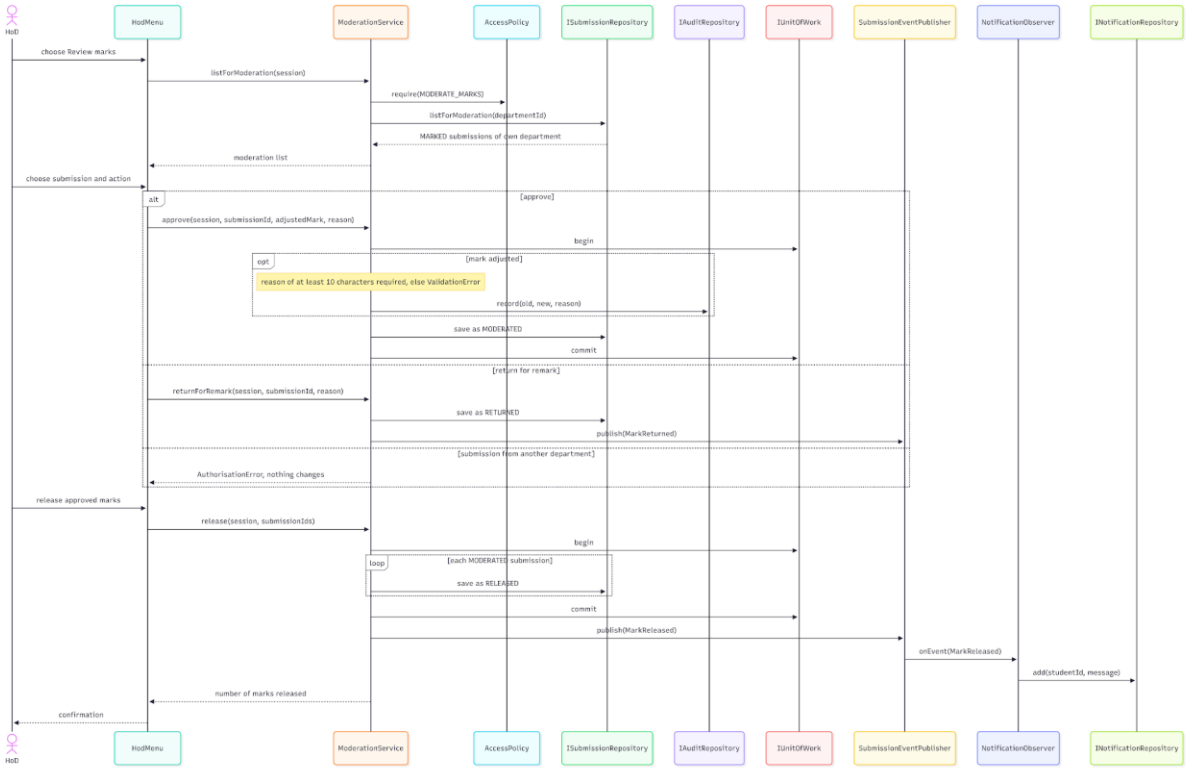


ii) Activity diagram

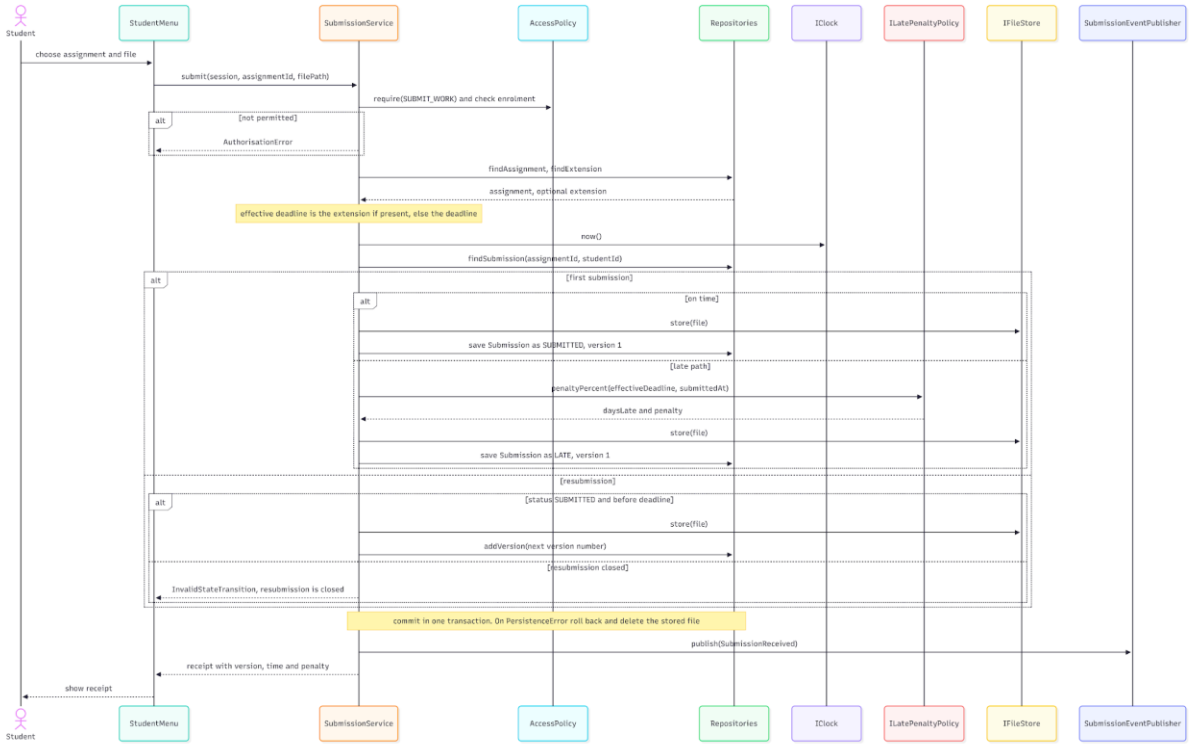


iii) Sequence diagram

Moderate and release marks

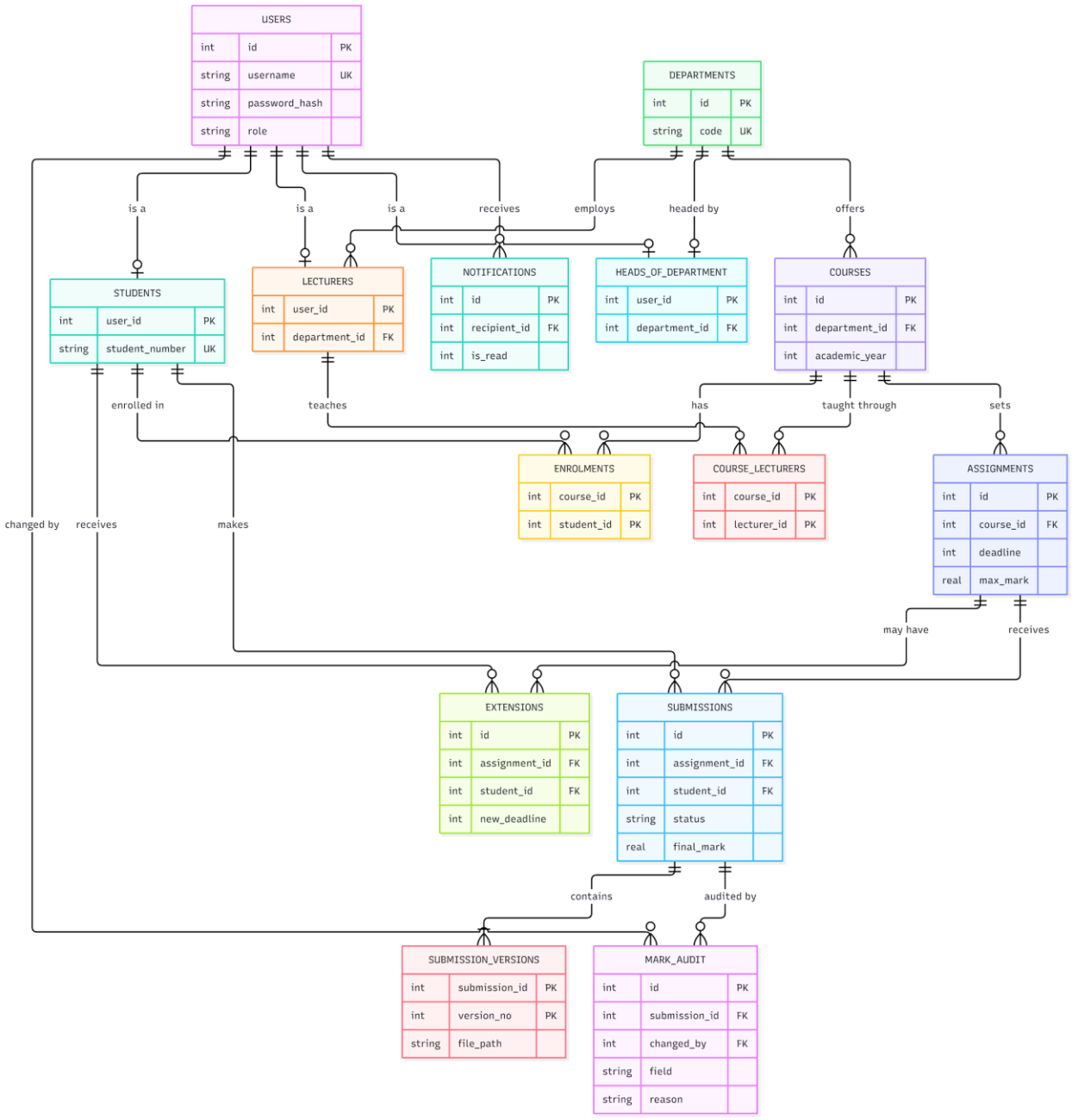


Submit assignment



3. Database design

i) ERD diagram



ii) Data dictionary

Table. Column	Type	Constraints	Description
users.id	INTEGER	PK, autoincrement	Unique user identifier
users.username	TEXT	UNIQUE, length 3–32	Login name
users.full_name	TEXT		Display name
users.email	TEXT	nullable	Contact address (personal data)
users.password_hash	TEXT		Encoded Argon2id hash including salt and parameters; never the password
users.role	TEXT	CHECK in ADMIN, LECTURER, HOD, STUDENT	Role that selects the User subclass
users.is_active	INTEGER	CHECK 0 or 1, default 1	Deactivated users cannot log in; records are kept
users.failed_attempts	INTEGER	default 0	Consecutive failed logins
users.locked_until	INTEGER	nullable	Lockout end time
users.created_at	INTEGER		Creation time
students.user_id	INTEGER	PK, FK users	Links the profile to the user
students.student_number	TEXT	UNIQUE	School-issued number
lecturers.user_id	INTEGER	PK, FK users	Links the profile to the user
lecturers.department_id	INTEGER	FK departments	Home department
heads_of_department.user_id	INTEGER	PK, FK users	Links the profile to the user; the user's role must be HOD (enforced by trigger)

heads_of_department.department_id	INTEGER	UNIQUE, FK departments	Department headed: one HoD per department, one department per HoD
departments.id	INTEGER	PK	Department identifier
departments.code	TEXT	UNIQUE	Short code
departments.name	TEXT	UNIQUE	Full name
courses.id	INTEGER	PK	Course identifier
courses.code	TEXT	UNIQUE with academic_year	Course code
courses.title	TEXT		Course title
courses.department_id	INTEGER	FK departments	Owning department
courses.academic_year	INTEGER		Year of offering
course_lecturers.course_id	INTEGER	PK part, FK courses	Course taught
course_lecturers.lecturer_id	INTEGER	PK part, FK lecturers	Lecturer assigned
enrolments.course_id	INTEGER	PK part, FK courses	Course enrolled in
enrolments.student_id	INTEGER	PK part, FK students	Enrolled student
enrolments.enrolled_at	INTEGER		Enrolment time
assignments.id	INTEGER	PK	Assignment identifier
assignments.course_id	INTEGER	FK courses	Parent course
assignments.title	TEXT		Title
assignments.description	TEXT	nullable	Instructions
assignments.deadline	INTEGER		Original deadline

assignments.max_mark	REAL	CHECK > 0	Maximum mark, the base for penalties
assignments.created_by	INTEGER	FK lecturers	Creating lecturer
assignments.created_at	INTEGER		Creation time
extensions.id	INTEGER	PK	Extension identifier
extensions.assignment_id	INTEGER	FK assignments	Assignment extended
extensions.student_id	INTEGER	FK students	Student who receives it
extensions.new_deadline	INTEGER		Student's effective deadline
extensions.reason	TEXT	length at least 5	Why it was granted
extensions.granted_by	INTEGER	FK lecturers	Granting lecturer
extensions.granted_at	INTEGER		When granted
submissions.id	INTEGER	PK	Submission identifier
submissions.assignment_id	INTEGER	FK assignments, UNIQUE with student_id	Assignment submitted to
submissions.student_id	INTEGER	FK students	Submitting student
submissions.status	TEXT	CHECK in the six states	Lifecycle state
submissions.submitted_at	INTEGER		Time of first submission
submissions.days_late	INTEGER	CHECK >= 0, default 0	Started days past the effective deadline
submissions.penalty_percent	REAL	CHECK 0–100, default 0	Penalty applied at submission
submissions.raw_mark	REAL	nullable, CHECK >= 0	Lecturer's mark before penalty

submissions.final_mark	REAL	nullable, CHECK >= 0	Mark after penalty and moderation
submissions.feedback	TEXT	nullable	Written feedback
submissions.updated_at	INTEGER		Last change
submission_versions.submission_id	INTEGER	PK part, FK submissions, cascade delete	Parent submission
submission_versions.version_no	INTEGER	PK part, CHECK >= 1	Version counter
submission_versions.file_path	TEXT		Stored file location
submission_versions.original_name	TEXT		Uploaded file name
submission_versions.size_bytes	INTEGER	CHECK >= 0	File size
submission_versions.uploaded_at	INTEGER		Upload time
mark_audit.id	INTEGER	PK	Audit entry identifier
mark_audit.submission_id	INTEGER	FK submissions	Submission affected
mark_audit.changed_by	INTEGER	FK users	Who changed the mark
mark_audit.changed_at	INTEGER		When
mark_audit.field	TEXT	CHECK in RAW_MARK, FINAL_MARK	Which mark changed
mark_audit.old_value	REAL	nullable	Previous value (null on first mark)
mark_audit.new_value	REAL		New value
mark_audit.reason	TEXT	length at least 1	Why
notifications.id	INTEGER	PK	Notification identifier
notifications.recipient_id	INTEGER	FK users	Recipient

notifications.message	TEXT		Message text
notifications.created_at	INTEGER		Creation time
notifications.is_read	INTEGER	CHECK 0 or 1, default 0	Read flag

iii) SQL schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL UNIQUE CHECK (length(username) BETWEEN 3 AND 32),
  full_name TEXT NOT NULL,
  email TEXT,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('ADMIN','LECTURER','HOD','STUDENT')),
  is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)),
  failed_attempts INTEGER NOT NULL DEFAULT 0,
  locked_until INTEGER,
  created_at INTEGER NOT NULL
);

CREATE TABLE departments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  code TEXT NOT NULL UNIQUE,
  name TEXT NOT NULL UNIQUE
);

CREATE TABLE students (
  user_id INTEGER PRIMARY KEY REFERENCES users(id),
  student_number TEXT NOT NULL UNIQUE
);

CREATE TABLE lecturers (
  user_id INTEGER PRIMARY KEY REFERENCES users(id),
  department_id INTEGER NOT NULL REFERENCES departments(id)
);

CREATE TABLE heads_of_department (
  user_id INTEGER PRIMARY KEY REFERENCES users(id),
  department_id INTEGER NOT NULL UNIQUE REFERENCES departments(id)
);
```

```
CREATE TABLE courses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  code TEXT NOT NULL,
  title TEXT NOT NULL,
  department_id INTEGER NOT NULL REFERENCES departments(id),
  academic_year INTEGER NOT NULL,
  UNIQUE (code, academic_year)
);

CREATE TABLE course_lecturers (
  course_id INTEGER NOT NULL REFERENCES courses(id),
  lecturer_id INTEGER NOT NULL REFERENCES lecturers(user_id),
  PRIMARY KEY (course_id, lecturer_id)
);

CREATE TABLE enrolments (
  course_id INTEGER NOT NULL REFERENCES courses(id),
  student_id INTEGER NOT NULL REFERENCES students(user_id),
  enrolled_at INTEGER NOT NULL,
  PRIMARY KEY (course_id, student_id)
);

CREATE TABLE assignments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  course_id INTEGER NOT NULL REFERENCES courses(id),
  title TEXT NOT NULL,
  description TEXT,
  deadline INTEGER NOT NULL,
  max_mark REAL NOT NULL CHECK (max_mark > 0),
  created_by INTEGER NOT NULL REFERENCES lecturers(user_id),
  created_at INTEGER NOT NULL
);
```

```
CREATE TABLE extensions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  assignment_id INTEGER NOT NULL REFERENCES assignments(id),
  student_id INTEGER NOT NULL REFERENCES students(user_id),
  new_deadline INTEGER NOT NULL,
  reason TEXT NOT NULL CHECK (length(reason) >= 5),
  granted_by INTEGER NOT NULL REFERENCES lecturers(user_id),
  granted_at INTEGER NOT NULL,
  UNIQUE (assignment_id, student_id)
);

CREATE TABLE submissions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  assignment_id INTEGER NOT NULL REFERENCES assignments(id),
  student_id INTEGER NOT NULL REFERENCES students(user_id),
  status TEXT NOT NULL CHECK (status IN
    ('SUBMITTED','LATE','MARKED','RETURNED','MODERATED','RELEASED')),
  submitted_at INTEGER NOT NULL,
  days_late INTEGER NOT NULL DEFAULT 0 CHECK (days_late >= 0),
  penalty_percent REAL NOT NULL DEFAULT 0 CHECK (penalty_percent BETWEEN 0 AND 100),
  raw_mark REAL CHECK (raw_mark >= 0),
  final_mark REAL CHECK (final_mark >= 0),
  feedback TEXT,
  updated_at INTEGER NOT NULL,
  UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission_versions (
  submission_id INTEGER NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
  version_no INTEGER NOT NULL CHECK (version_no >= 1),
  file_path TEXT NOT NULL,
  original_name TEXT NOT NULL,
  size_bytes INTEGER NOT NULL CHECK (size_bytes >= 0),
  uploaded_at INTEGER NOT NULL,
  PRIMARY KEY (submission_id, version_no)
);
```

```
CREATE TABLE mark_audit (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  submission_id INTEGER NOT NULL REFERENCES submissions(id),
  changed_by INTEGER NOT NULL REFERENCES users(id),
  changed_at INTEGER NOT NULL,
  field TEXT NOT NULL CHECK (field IN ('RAW_MARK','FINAL_MARK')),
  old_value REAL,
  new_value REAL NOT NULL,
  reason TEXT NOT NULL CHECK (length(reason) >= 1)
);

CREATE TABLE notifications (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  recipient_id INTEGER NOT NULL REFERENCES users(id),
  message TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  is_read INTEGER NOT NULL DEFAULT 0 CHECK (is_read IN (0,1))
);

-- Indexes for the busiest queries (login, lists, moderation queue)
CREATE INDEX idx_courses_department ON courses(department_id);
CREATE INDEX idx_enrol_student ON enrolments(student_id);
CREATE INDEX idx_assign_course ON assignments(course_id);
CREATE INDEX idx_sub_student ON submissions(student_id);
CREATE INDEX idx_sub_assign_status ON submissions(assignment_id, status);
CREATE INDEX idx_audit_submission ON mark_audit(submission_id);
CREATE INDEX idx_notif_user ON notifications(recipient_id, is_read);

-- The audit log is append-only, enforced by the database itself
CREATE TRIGGER audit_no_update BEFORE UPDATE ON mark_audit
BEGIN SELECT RAISE(ABORT, 'audit log is append-only'); END;
CREATE TRIGGER audit_no_delete BEFORE DELETE ON mark_audit
BEGIN SELECT RAISE(ABORT, 'audit log is append-only'); END;

-- Only a user whose role is HOD may be made head of a department
CREATE TRIGGER hod_role_insert BEFORE INSERT ON heads_of_department
WHEN (SELECT role FROM users WHERE id = NEW.user_id) <> 'HOD'
BEGIN SELECT RAISE(ABORT, 'user is not a HOD'); END;
```

```
CREATE TRIGGER hod_role_update BEFORE UPDATE OF user_id ON heads_of_department
WHEN (SELECT role FROM users WHERE id = NEW.user_id) <> 'HOD'
BEGIN SELECT RAISE(ABORT, 'user is not a HOD'); END;
```

iv) Mapping from classes to tables, and justified differences

Class	Table	Difference and justification
User and its four subclasses	users plus students, lecturers, heads_of_department	Class-table inheritance. Common data sits in users; role-specific data in one profile table per role (students, lecturers, heads_of_department; the Administrator needs none). Single-table inheritance would force nullable student_number and department_id columns on every user. Concrete-table inheritance would duplicate credentials and break foreign keys such as mark_audit.changed_by. UserFactory joins the tables to build the right subclass.
HeadOfDepartment (including departmentId)	heads_of_department	Same class-table pattern as Student and Lecturer. The primary key user_id and the UNIQUE department_id give one department per HoD and at most one HoD per department. A trigger rejects any user whose role is not HOD. No difference from the class: departmentId maps straight to the column.
Role enum	users.role	Stored as text with a CHECK, so the database rejects an unknown role.
Department, Course, Assignment, Extension, Notification	same-named tables	One-to-one. TimePoint becomes epoch seconds.
Enrolment, CourseLecturer (link records)	enrolments, course_lecturers	Many-to-many associations become tables with composite keys.

Submission	submissions	currentVersion is not stored. It is derived as the highest version_no, so it cannot disagree with the versions table. penaltyPercent, daysLate and finalMark are stored on purpose (section 5.2).
SubmissionVersion	submission_versions	Composite key; the file itself is on disk, not in the database.
AuditEntry	mark_audit	One row per changed field, so a mark and penalty change produces two rows. Append-only by trigger.
SubmissionStateMachine, penalty policies, services	none	Behaviour only, nothing to persist.